

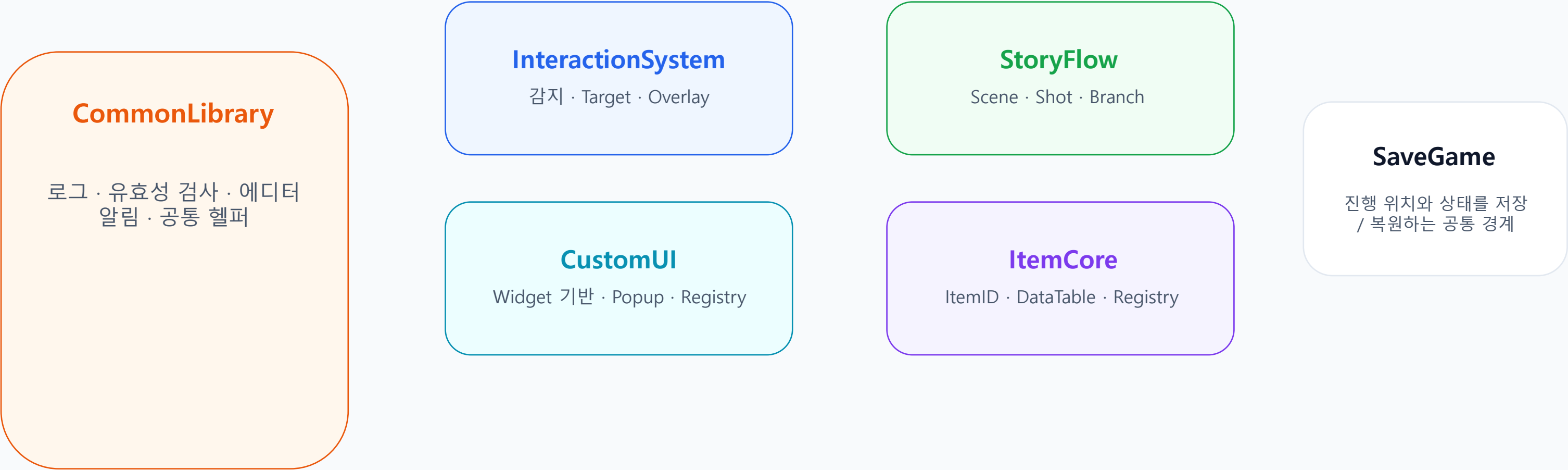
플러그인 기반 클라이언트 시스템 포트폴리오

상호작용 · StoryFlow · 3D Monitor UI · Data / Save Pipeline

본 자료는 콘텐츠 서술보다 UE5 C++ 클라이언트 시스템 설계와 구현 판단을 중심으로 정리했습니다.

플러그인 기반 클라이언트 구조

게임 본체는 조립부로 두고, 반복 사용되는 기능은 플러그인과 서브시스템 경계로 분리했습니다.



공통 기반 위에 기능 플러그인을 쌓고, 프로젝트 본체는 필요한 기능을 조립합니다.

그래프 기반 진행 시스템

진행 단위를 Scene / Shot / Branch로 분리해 편집, 검증, 런타임 실행 경계를 만들었습니다.



시연 영상: 게임 시작 → 로비 진입
https://www.youtube.com/watch?v=V0NzNsPZ_LA

왜 만들었나

진행 흐름이 레벨과 개별 위젯에 흩어지면 수정·검증 비용이 커집니다.

어떻게 구현했나

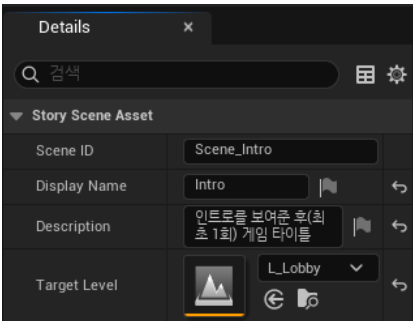
편집 가능한 그래프를 만들고, 실행 전 검증한 뒤 런타임에서는 확인된 흐름만 소비하도록 나눴습니다.

얻는 이점

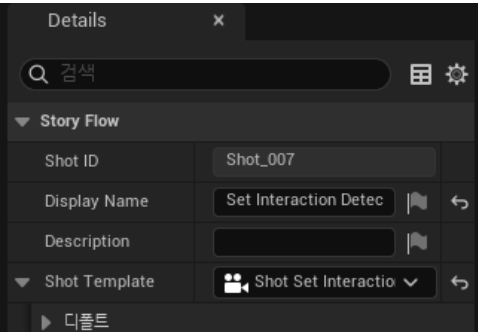
진행 구조를 시각적으로 확인하고, 특정 지점부터 테스트하며, 이후 콘텐츠 확장에도 같은 실행 규칙을 유지할 수 있습니다.

에디터 제작 경계와 런타임 실행 경계

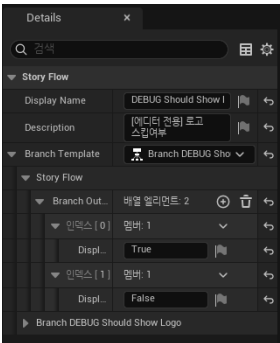
에디터에서는 데이터와 연결을 검증하고, 런타임에서는 현재 진행 위치를 기준으로 실행합니다.



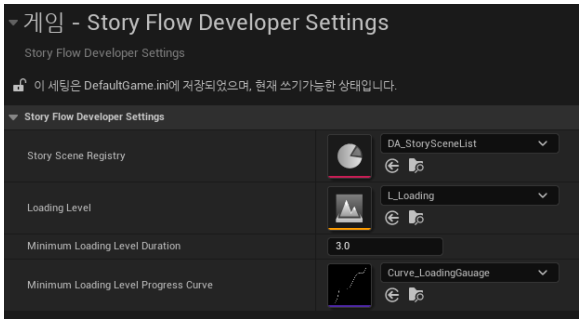
Scene 설정



Shot 설정



Branch 설정



전역 설정

그래프 편집

연결 검증

실행 데이터

현재 위치 저장

지점 재시작

왜 분리했나

제작자는 그래프와 에셋을 다루고, 런타임은 검증된 실행 단위만 처리하게 하기 위해서입니다.

이점

오류를 실행 전에 찾고, 특정 진행 지점부터 재현하는 테스트 흐름을 만들 수 있습니다.

포트폴리오 메시지

콘텐츠 서술보다 “진행을 도구화한 클라이언트 시스템”으로 설명합니다.

상호작용을 Actor별 구현이 아닌 공통 시스템으로 분리

감지, 선택, 피드백, 실행 호출을 컴포넌트와 인터페이스 경계로 묶었습니다.



왜 만들었나

상호작용 대상이 늘어날수록 Actor마다 감지와 표시 로직을 반복 작성하는 문제가 생깁니다.

어떻게 구현했나

상호작용 주체는 공통 감지 컴포넌트를 갖고, 대상 Actor는 약속된 인터페이스만 구현하도록 분리했습니다.

얻는 이점

컴퓨터, 조사 오브젝트, 아이템 등 서로 다른 대상도 같은 감지·선택·피드백 규칙으로 확장할 수 있습니다.

카메라 중심 감지와 커서 기반 감지를 분리해, 1인칭 탐색과 모니터 조작 상황을 모두 수용합니다.

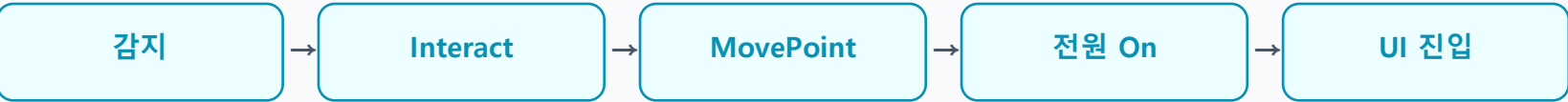
월드 안 컴퓨터를 조작하는 디제틱 UX

메뉴를 따로 띄우는 대신, 월드 오브젝트와 플레이어 이동, 모니터 화면을 하나의 흐름으로 연결했습니다.



컴퓨터 Actor 캡처

진입 흐름



Computer Actor

장치 상태, 화면 출력, 상호작용 진입점을 담당합니다.

Pawn / Input

이동 지점, 카메라 상태, 입력 모드와 UI 조작 전달을 담당합니다.

왜 만들었나

플레이어가 게임 밖 메뉴가 아니라 세계 안 장치를 직접 조작한다고 느끼게 하기 위해서입니다.

어떻게 구현했나

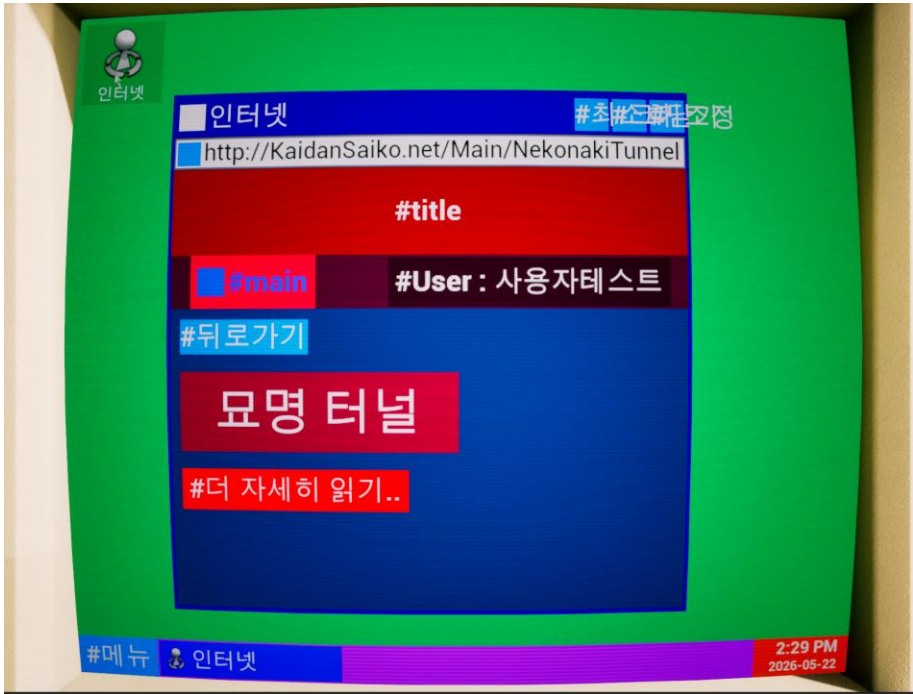
월드 Actor와 Pawn의 책임을 나누고, 상호작용 후 이동·전원·UI 진입을 순서대로 연결했습니다.

얻는 이점

상호작용, 카메라, 입력 모드, UI 표시가 하나의 자연스러운 사용자 흐름으로 묶입니다.

3D 표면 Hit를 2D UMG 입력으로 보정

월드 좌표, 위젯 로컬 좌표, 커서 좌표의 차이를 보정해 Hover / Click / Scroll을 같은 경로로 전달했습니다.



3D 모니터 위젯 실행 화면



문제

모니터 외곽이나 카메라 각도에 따라 커서 위치와 실제 UMG 입력 위치가 어긋날 수 있습니다.

해결

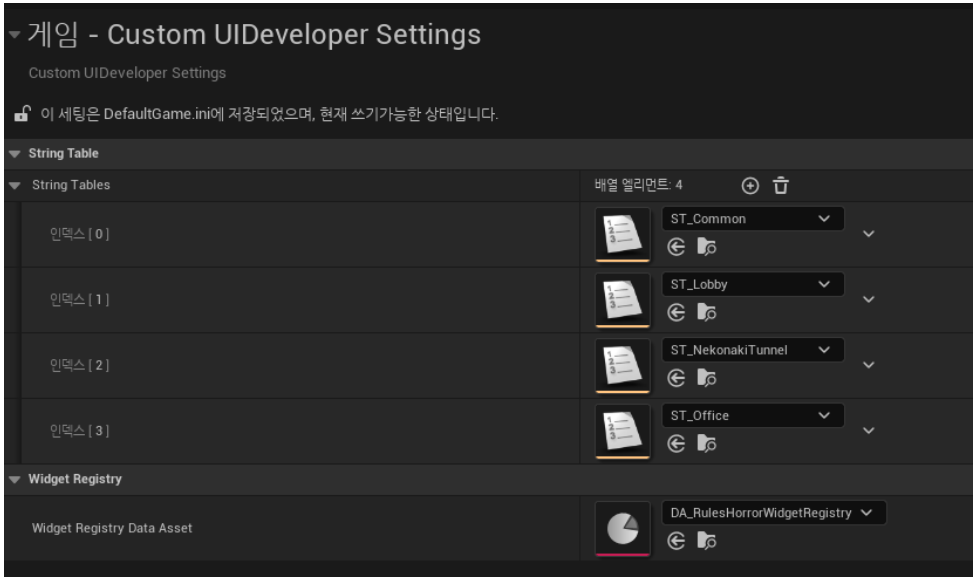
표면 Hit를 위젯 기준 좌표로 다시 계산하고, 보정된 결과를 위젯 입력 경로에 주입했습니다.

시연 영상: 3D 모니터 / 위젯 기능

<https://www.youtube.com/watch?v=BlseQIZLjQs>

공통 UI 플러그인 위에 모니터 UI를 조립

위젯 표시 상태, 팝업, Registry, 버튼 계열을 공통화해 프로젝트 UI가 같은 규칙을 쓰도록 했습니다.



CustomUI 설정 캡처

왜 만들었나

UI마다 열기/닫기, 애니메이션, 팝업 정리, 텍스트 조회 규칙을 따로 만들면 유지보수 비용이 커집니다.

어떻게 구현했나

위젯 기본 상태, 등록 위젯 조회, 팝업 스택, 버튼 계열 컴포넌트를 플러그인 경계로 묶었습니다.

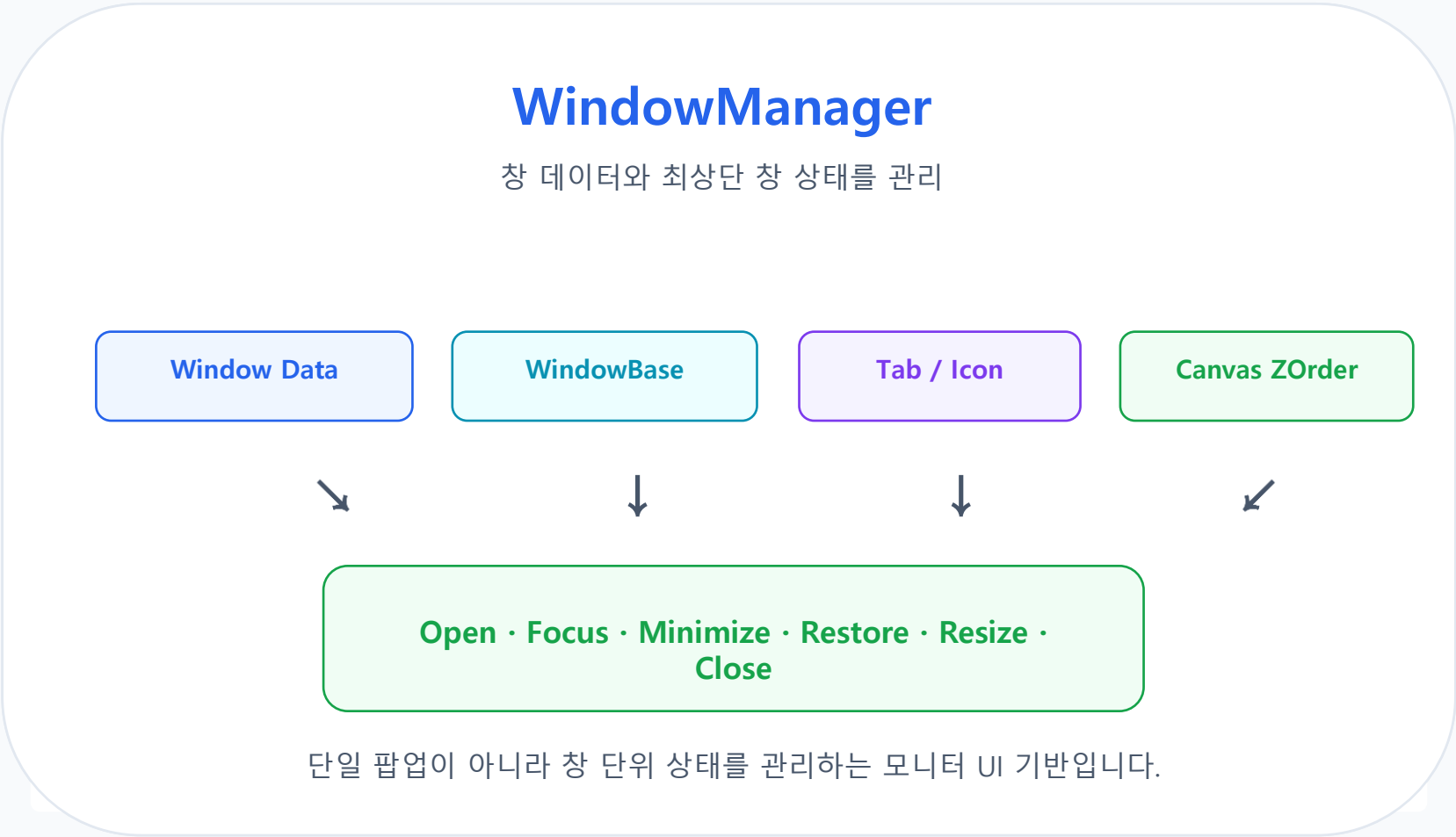
얻는 이점

모니터 UI, 팝업, 버튼, 텍스트 연출을 같은 기반 위에서 추가할 수 있습니다.



모니터 내부는 단일 화면이 아니라 창 시스템

창, 탭, 아이콘, 포커스, ZOrder, 드래그/리사이즈 상태를 묶어 데스크톱형 UX를 구성했습니다.



- 왜 만들었나

모니터 안에서 여러 사이트와 창을 다루려면 단순 화면 전환보다 창 단위 상태 관리가 필요했습니다.
- 어떻게 구현했나

창 데이터, 탭, 아이콘, 최상단 창, 최소화/복원/최대화, 이동/리사이즈 명령을 공통 관리합니다.
- 얻는 이점

새 앱이나 사이트형 UI를 추가해도 창 관리 규칙을 다시 만들 필요가 줄어듭니다.

창 생성

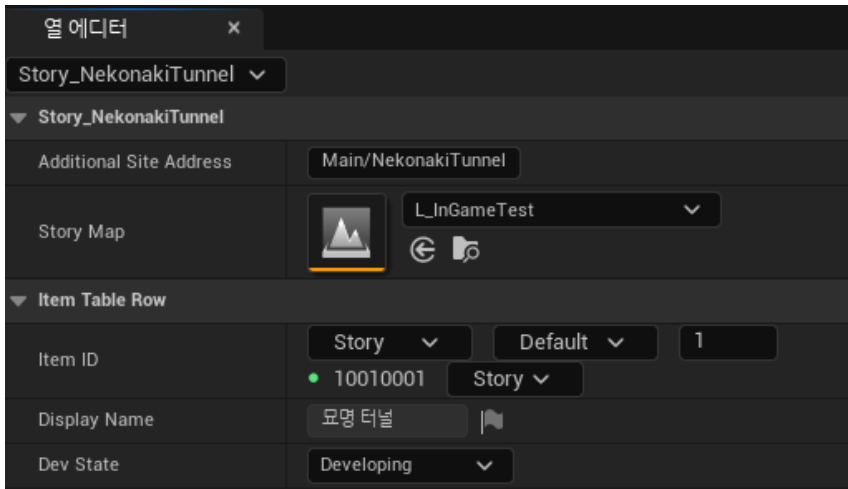
포커스

탭

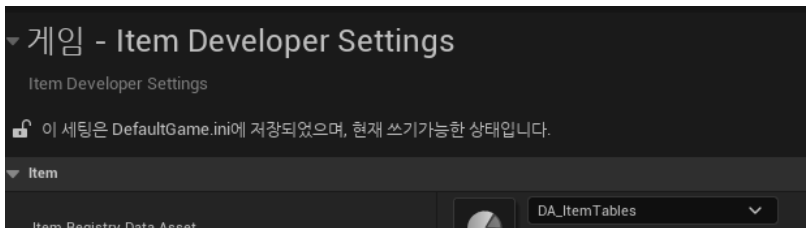
리사이즈

DataTable / Registry 기반 콘텐츠 데이터 구조

UI가 맵이나 콘텐츠 정보를 직접 들고 있지 않고, 식별자와 Registry를 통해 필요한 데이터를 조회합니다.



Story Item Row 캡처



Item 설정 캡처

왜 만들었나

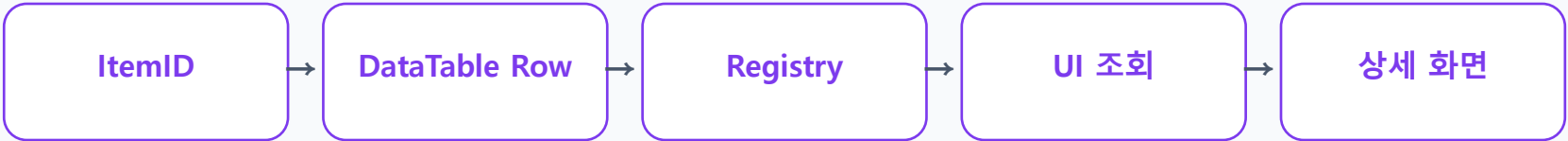
콘텐츠 항목이 늘어날수록 UI와 Actor에 데이터를 직접 연결하면 누락과 중복을 찾기 어렵습니다.

어떻게 구현했나

ItemID, DataTable Row, Registry DataAsset, Editor Picker를 통해 데이터 등록과 조회 경계를 만들었습니다.

얻는 이점

새 콘텐츠를 추가할 때 코드 수정보다 데이터 추가와 Registry 검증을 우선할 수 있습니다.



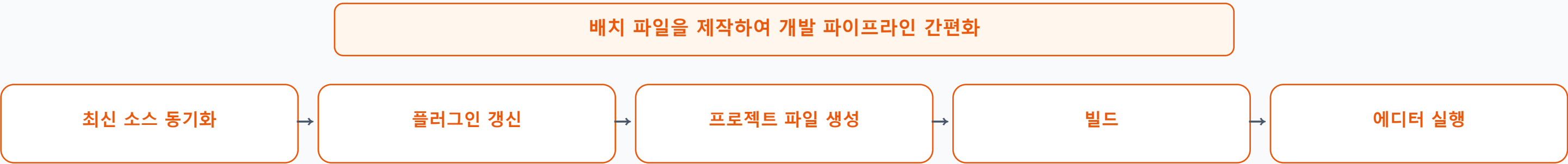
진행 위치와 저장 정책을 플러그인 경계로 분리

공통 저장 슬롯, key-value 데이터, 비동기 저장/로드 정책 위에 프로젝트별 진행 정보를 얹었습니다.



반복 개발 루틴 자동화

동기화, 프로젝트 파일 생성, 빌드, 실행처럼 반복되는 절차를 묶어 작업 전환 비용을 줄였습니다.



왜 만들었나

UE 프로젝트는 플러그인 갱신, 프로젝트 파일 재생성, 빌드/실행 반복이 자주 발생합니다.

어떻게 구현했나

Windows 환경에서 자주 쓰는 절차를 배치 루틴으로 묶고, 작업별 책임을 나눴습니다.

얻는 이점

테스트 시작까지의 수동 단계를 줄이고, 브랜치/플러그인 동기화 누락 가능성을 낮춥니다.

배치 파일을 제작하여 개발 파이프라인을 간편화하고, 동기화부터 실행까지 반복 절차를 한 흐름으로 묶었습니다.

핵심 역량: 클라이언트 시스템 경계 설계

이 포트폴리오는 콘텐츠 서술보다 기능을 나누고, 연결하고, 검증 가능한 형태로 만든 과정을 보여줍니다.

핵심 역량 3줄 요약

플러그인 기반 구조 설계

게임 본체와 재사용 가능한 기능을 분리해, 상호작용·UI·데이터·저장·진행 시스템을 독립적인 경계로 구성했습니다.

월드와 UI를 연결하는 클라이언트 구현

월드 오브젝트 상호작용, 3D 모니터 입력 보정, UMG 창 시스템을 하나의 사용자 흐름으로 연결했습니다.

데이터 기반 제작 파이프라인 구축

StoryFlow, Item Registry, SaveGame, 배치 자동화를 통해 콘텐츠 추가와 반복 테스트가 쉬운 개발 기반을 만들었습니다.